



**UNIVERSIDAD AUTÓNOMA
DE AGUASCALIENTES**

Ingeniería en Sistemas Computacionales

8B

Seguridad en Sistemas

Primeros pasos cifrando

Docente:

Arturo Ocampo Silva

Alumno:

Oswaldo De La Cruz García

Fecha de entrega:

Viernes 20 de febrero de 2026

Índice

Introducción.....	2
Objetivo.....	2
Desarrollo.....	3
Conclusión.....	7
Bibliografías.....	7

Introducción

En el campo de la seguridad informática, el cifrado de datos es fundamental para proteger la información sensible durante su transmisión o almacenamiento. A lo largo de la historia se han desarrollado diferentes técnicas de cifrado para asegurar que los mensajes sean legibles solo por las personas autorizadas. Dos de los métodos más antiguos de cifrado son el cifrado César y el Cifrado Atbash.

El Cifrado César es una técnica simple en la que cada letra del alfabeto se sustituye por otra que se encuentra en número fijo de posiciones más adelante en el alfabeto. Por ejemplo, con un desplazamiento de 3, la letra "A" se convierte en "D", "B" en "E", y así sucesivamente. Este método fue utilizado por Julio César para proteger sus mensajes y es uno de los ejemplos más antiguos de cifrado conocido.

El Cifrado Atbash por otro lado, es un tipo de cifrado de sustitución en el cual las letras del alfabeto se invierten. Es decir, "A" se convierte en "Z", "B" en "Y" y así sucesivamente. Este método es muy antiguo y su simplicidad lo hacía fácil de usar en tiempos pasados.

Aunque ambos métodos fueron efectivos en su época, hoy en día se consideran inseguros debido a su simplicidad. Los avances tecnológicos han permitido a los atacantes realizar análisis rápidos y eficientes de los cifrados, lo que hace que estos métodos sean fácilmente descifrables mediante ataques de fuerza bruta o análisis de frecuencia. Los sistemas de cifrado, permitiendo a los usuarios cifrar y descifrar mensajes utilizando César o Atbash. Además, se pretende demostrar cómo estas técnicas pueden ser útiles para entender los principios básicos de la criptografía, a pesar de que ya no son adecuadas. Para aplicaciones de seguridad modernas.

Objetivo

El objetivo de este proyecto es desarrollar un programa que permita cifrar y descifrar mensajes utilizando los métodos de cifrado clásicos César y Atbash. El programa debe ser fácil de usar y permitir al usuario seleccionar el método de cifrado deseado. Además, se demostrará cómo estos métodos, aunque simples, pueden ser útiles para entender los principios básicos de la criptografía, a pesar de no ser seguros para aplicaciones modernas debido a su vulnerabilidad ante ataques de fuerza bruta.

Desarrollo

Para este proyecto se seleccionaron las herramientas que permiten rapidez de despliegue, flexibilidad lógica y una interfaz moderna.

- **Python 3.x:** Elegido por su robustez en el manejo de cadenas de texto y bibliotecas de lógica matemática. Es el lenguaje estándar en ciberseguridad para prototipado de herramientas criptográficas.
- **Streamlit:** Se utilizó para la interfaz web debido a su capacidad de crear dashboards interactivos sin la sobrecarga de un framework fronted pesado como react o angular. Esto cumple con el requisito de “Programa en web” de forma ágil.
- **Git/Github:** Utilizados para el control de versiones. Esto garantiza la trazabilidad del código (saber qué cambió y cuando) y la integridad del desarrollo (asegurar que el código entregado es el original).

Arquitectura del motor lógico (cifrado.py)

El motor de la aplicación no utiliza tablas estáticas de conversión, sino que genera alfabetos dinámicos en tiempo real.

A. Gestión de alfabetos y aritmética modular

El sistema calcula el índice de cada carácter basándose en la posición dentro de una lista dinámica.

- Modo estándar que utiliza un set de caracteres basado en el español incluyendo la ñ, lo que expande el módulo de calculo a $n = 28$ incluyendo espacios
- Modo personalizado que permite al usuario alimentar el sistema con un conjunto de caracteres específicos. Esto es fundamental para romper la limitación de los cifrados clásicos que solo funcionan con letras.

B. Algoritmos de transformación

- **Cifrado César:** Implementa un desplazamiento circular. Si el desplazamiento supera la longitud del alfabeto, se aplica la operación del modulo x para asegurar que el puntero siempre caiga dentro del rango definido.
- **Cifrado Atbash:** Funciona como un espejo matemático. Si el índice de un carácter es i en un alfabeto de longitud n , su equivalente cifrado será.

Desarrollo del “Ingenio”: Motor de detección de estadística

El mayor valor agregado de este desarrollo es el Análisis Automático, que utiliza principios de criptoanálisis para descifrar mensajes sin que el usuario conozca la clave.

Lógico de Puntuación

El sistema ejecuta todas las combinaciones posibles de César y la opción de Atbash, evaluando el resultado de cada una mediante un sistema de puntos:

- **Análisis de frecuencia de letras:** Basado en los estudios de Al-Kindi, el programa otorga puntos si el resultado contiene letras frecuentes en español (E, A, O, S, R).
- **Reconocimiento de Patrones:** Se implementó un diccionario de palabras clave técnicas. Si una palabra coincide, el mensaje recibe un bonus masivo de confianza.
- **Coherencia de Caracteres:** En el modo personalizado, el sistema premia los resultados que mantienen la misma naturaleza (letras o números) que el mensaje original, evitando falsos positivos.

Manual de implementación y Uso

Paso 1: Configuración del entorno

El usuario decide si trabajará con el abecedario español o con un alfabeto personalizado

Paso 2: Entrada de Datos

El texto ingresado es procesado para eliminar espacios extra si es necesario, pero manteniendo la estructura original para no corromper el mensaje.

Paso 3: Análisis y selección

- **Manual:** El usuario usa los botones de Cifrar César o Atbash para pruebas rápidas
- **Automático:** El botón Ejecutar análisis automático, dispara el motor de criptoanálisis. El sistema despliega una Consola de Resultados comparativa donde muestra la confianza de cada método.

Documentación del código fuente

1. Estructura de Datos y Alfabetos Dinámicos

El programa no utiliza una tabla de consulta estática, lo que permite la flexibilidad de "alimentar" el sistema con cualquier conjunto de caracteres.

- **obtener_alfabetos:** Esta función es el corazón de la flexibilidad del sistema. Si el usuario activa el "Modo Personalizado", la función toma la cadena de entrada, elimina caracteres duplicados y genera un alfabeto único.
- **Manejo de la Ñ:** En el modo estándar, se utiliza una cadena predefinida que incluye la "ñ" para cumplir con las reglas del idioma español, ajustando el módulo matemático a 28.

2. Implementación de los Algoritmos de Cifrado

Ambos algoritmos se basan en la manipulación de índices dentro del alfabeto generado.

A. Cifrado César (Desplazamiento Circular)

def cifrar_cesar(texto, desplazamiento, alfabeto_custom, con_enie):

n es el tamaño del alfabeto (ej. 28 para español o len(custom))

Se aplica la fórmula: (índice + desplazamiento) % n

- **Lógica:** Se crea un "alfabeto espejo". Debido a que la reflexión es recíproca, la misma función sirve tanto para cifrar como para descifrar.

3. Motor de Análisis Estadístico (Criptoanálisis)

Esta es la sección de "ingenio" que analiza la coherencia del lenguaje.

- **calcular_puntuacion_idioma:** Evalúa la calidad de un descifrado asignando pesos numéricos.
 - **Palabras Clave:** Otorga **+2000 puntos** si encuentra términos técnicos como "circuitos" o "rectificadores".
 - **Frecuencia de Letras:** Suma puntos por la presencia de letras comunes en español (E, A, O, S, R), permitiendo identificar frases reales incluso sin palabras del diccionario.
 - **Coherencia de Tipo:** En modo personalizado, premia los resultados que mantienen la estructura (si el original tenía números, el descifrado también debería tenerlos).

4. Interfaz Web (Capa de Presentación)

Desarrollada en app.py utilizando **Streamlit**.

- **st.sidebar:** Gestiona la configuración global (Modo Custom vs. Ñ) sin recargar la página completa.
- **st.columns:** Implementa la **Comparativa de Resultados**, permitiendo al usuario visualizar simultáneamente el candidato de Atbash y el mejor candidato de César para decidir cuál tiene más sentido.

Conclusión

El desarrollo demuestra que los cifrados César y Atbash, aunque históricamente importantes, son totalmente vulnerables al análisis computacional moderno. Este software no solo cifra información, sino que sirve como una herramienta pedagógica para demostrar por qué la criptografía asimétrica moderna es necesaria hoy en día.

Bibliografías

- Al-Kadi, I. A. (1992). Origins of cryptology: The Arab contributions. *Cryptologia*, 16(2), 97-126. <https://doi.org/10.1080/0161-1109291886866>
- Kahn, D. (1996). *The codebreakers: The comprehensive history of secret communication from ancient times to the internet*. Scribner.
- Singh, S. (1999). *The code book: The science of secrecy from ancient Egypt to quantum cryptography*. Fourth Estate.
- Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7^a ed.). Pearson.
- Chacon, S., y Straub, B. (2014). *Pro Git* (2^a ed.). Apress. <https://git-scm.com/book/en/v2>
- Python Software Foundation. (2026). *Python language reference, version 3.12*. <https://docs.python.org/3/>
- Streamlit Inc. (2026). *Streamlit documentation: The fastest way to build and share data apps*. <https://docs.streamlit.io/>